# Dakopen's PACE 2026 Solver (Heuristic Track)

Daniel Busch
Student at the University of Tübingen, Germany
dakopen185@gmail.com

**Preliminary leaderboard result:** 99.82 / 100 points on the optil.io Heuristic Track.

**Abstract**

This solver combines optimum-preserving data reductions with a set-packing ILP over valid agreement components. Since the number of valid components is exponential, the ILP is solved heuristically by column generation, where a weighted agreement-subtree dynamic program acts as the pricing oracle. A limited branch-and-price phase is then used to improve fractional LP solutions when time permits.

## 1. Introduction

Given two rooted, unordered binary trees $T_1$ and $T_2$ with the same leaf-set $X$, a so-called **Maximum Agreement Forest** (MAF) is a partition of $X$ into the fewest possible components such that each component induces isomorphic subtrees in both trees, and these induced subtrees are edge-disjoint within each input tree. The question of whether a MAF for $T_1$ and $T_2$ with $k+1$ components exists was proven to be NP-hard by Hein et al. [1]. In this description I present an anytime heuristic solver based on an ILP set-packing formulation and column generation.

## 2. Methods

My algorithm combines several methods:

### 2.1. Data reduction

To begin with, I run these three optimum-preserving data reduction steps on both trees.

#### 2.1.1. Common Subtree Reduction

If both input trees contain an identical subtree, meaning a subtree with the same leaf set and isomorphic topology, this subtree can be replaced by a single new leaf without changing the optimal solution size [2].

#### 2.1.2. Common Chain Reduction

If both input trees contain the same common chain of length $j \geq 4$, this chain can be replaced in both trees by a representative chain of length 3 [2].

#### 2.1.3. 3-2-Chain Reduction

If one tree contains a pendant 3-chain $((x_1, x_2), x_3)$ while the other contains only one of the corresponding pendant 2-chains $(x_1, x_3)$ or $(x_2, x_3)$, the edge to the remaining leaf can be cut off [3].

### 2.2. Integer Linear Program

The core of my algorithm is a set-packing Integer Linear Program (ILP). Let a component $c \subseteq X$ be considered valid if its induced subtree is isomorphic in both trees. Now we want to choose as few as possible valid compo-

[1] If it does not appear in any chosen component, it is added as a singleton.

nents, under the two constraints that (1) each leaf is at most in one chosen component[1] and (2) no two chosen components use the same edge in either tree.

Let $x_c \in \{0, 1\}$ denote whether a component is chosen (1) or not chosen (0). Let $n$ be the number of leaves in each tree ($n = |X|$). Let $|c|$ be the number of leaves in each component. Specifically, our objective is to minimize

$$n - \sum((|c| - 1) \cdot x_c) \tag{1}$$

under the constraints

$$\forall l \in X : \quad \sum_{c \subseteq X : l \in c} x_c \leq 1 \quad \text{Every leaf is at most used in one chosen component} \tag{2}$$

and let $E$ be the set of all edges in both trees

$$\forall e \in E : \quad \sum_{c \subseteq X : e \in c} x_c \leq 1 \quad \text{Every edge at most used in one chosen component} \tag{3}$$

Leaves that are not covered by any chosen component become a singleton subtree in the resulting forest.

Since there are exponentially many valid components, I implemented a column generating algorithm similar to the exact MAF solver by Frohn et al. [4]. In short, I calculate the corresponding linear program dual value (*shadow prices*) and select new components (so-called *columns*) based on these prices and their gains with respect to the objective in Equation 1.

The condition that $x_c \in \{0, 1\}$ is then being relaxed in the corresponding linear program to $x_c \geq 0$ (which is because of the constraints equivalent to $x_c \in [0, 1]$). Our objective becomes:

$$\begin{aligned} \max \quad & \sum_c (|c| - 1) x_c \\ \text{s.t.} \quad & \sum_{c \ni l} x_c \leq 1 \quad \forall l \in X, \\ & \sum_{c \ni e} x_c \leq 1 \quad \forall e \in E, \\ & x_c \geq 0 \quad \forall c. \end{aligned} \tag{4}$$

The dual problem is

$$\begin{aligned} \min \quad & \sum_{l \in X} \lambda_l + \sum_{e \in E} \mu_e \\ \text{s.t.} \quad & \sum_{l \in c} \lambda_l + \sum_{e \in c} \mu_e \geq |c| - 1 \quad \forall \ \text{ valid components } \ c, \\ & \lambda_l \geq 0 \quad \forall l \in X, \\ & \mu_e \geq 0 \quad \forall e \in E. \end{aligned} \tag{5}$$

The values for each $l \in X$ in $\lambda_l$ and $e \in E$ in $\mu_e$ are called *shadow prices* as they tell us how "expensive" it is to use a certain edge or leaf (meaning the more components want to use it, the more expensive it becomes). Now we search for *columns* (new components), so we want to find missing valid components that would reduce the cost of the LP (= generate profit):

$$p(c) = (|c| - 1) - \sum_{l \in c} \lambda_l - \sum_{e \in c} \mu_e. \tag{6}$$

If $p(c) > 0$, then the current restricted LP is missing a component that can improve the relaxation, so the component is added as a new column.The pricing problem is therefore to find a valid agreement component with maximum positive profit. I solve this problem with a weighted maximum-agreement-subtree dynamic program over the two input trees. The leaf duals and edge duals are interpreted as penalties, while adding leaves gives reward according to $|c| - 1$. The dynamic program considers matching leaves, skipping a child in either tree, and combining two child matches either straight or crossed, which is necessary because the input trees are unordered. In practice, the implementation stores only relevant DP cells and uses a memory budget; if the budget

is exceeded, pricing falls back to the columns found so far. In each run of the pricer, my algorithm is collecting multiple profitable columns (depending on the size of the trees and a diversification factor).

### 2.3. Branch and Price

The pricing step gives an exact LP relaxation, but not necessarily an integer solution. In theory, branch and price turns this relaxation into an exact integer method, but considering the contest time limit of 5 minutes I only run it as an improvement heurstic.

If no component with positive profit exists, the restricted LP is optimal with respect to all valid components. The resulting LP value is an upper bound on the maximum achievable gain, and therefore gives a lower bound on the number of components in any MAF. However, the LP solution may still be fractional: it may assign, for example, half of one component and half of another. A valid agreement forest requires integral decisions $x_c \in \{0, 1\}$. For branching, I choose a fractional variable $x_c$ closest to $0.5$ and create two subproblems: one with $x_c = 1$ and one with $x_c = 0$. The first branch forces this component into the forest, while the second branch forbids it. In each branch, pricing is run again to generate columns that are compatible with the additional branching decision. The incumbent is the best integral solution found so far. A branch can be pruned if its lower bound is at least as large as the incumbent, because it cannot improve the current best solution. The process continues until an integral solution is found, all remaining branches are pruned, the time limit is reached, or a SIGTERM signal is received.

## References


[1] Jotun Hein, Tao Jiang, Lusheng Wang, and Kaizhong Zhang, "On the complexity of comparing evolutionary trees," *Discrete Applied Mathematics*, vol. 71, no. 1–3, pp. 153–169, 1996, doi: 10.1016/S0166-218X(96)00062-5.

[2] Magnus Bordewich and Charles Semple, "On the Computational Complexity of the Rooted Subtree Prune and Regraft Distance," *Annals of Combinatorics*, vol. 8, no. 4, pp. 409–423, 2005, doi: 10.1007/s00026-004-0229-z.

[3] Steven Kelk, Simone Linz, and Ruben Meuwese, "Cyclic Generators and an Improved Linear Kernel for the Rooted Subtree Prune and Regraft Distance," *Information Processing Letters*, vol. 180, p. 106336, 2023, doi: 10.1016/j.ipl.2022.106336.

[4] Martin Frohn, Steven Kelk, and Simona Vychytilova, "A branch-&-price approach to the unrooted maximum agreement forest problem," *Operations Research Letters*, vol. 63, p. 107364, 2025, doi: 10.1016/j.orl.2025.107364.